

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester IV)
Artificial Intelligence

Practical – I

Roll No. : T014	Name: Parth R. Kunkunkar
Class : TYIT	Date/Time of Submission : 27/06/2026

Aim: Introduction to AI and Intelligent Agents.

1). Design a Simple Rational Agent by defining PEAS for Vacuum Cleaner Environment. Classify environments (fully/partially observable, deterministic/stochastic, episodic/sequential). Implement a simple table-driven agent in Python.

→

CODE:

```
table = {
    ('A', 'Dirty'): 'Suck',
    ('A', 'Clean'): 'Right',
    ('B', 'Dirty'): 'Suck',
    ('B', 'Clean'): 'Left'
}

sequence_table = {
    (('A', 'Dirty'), ('A', 'Clean')): 'Right',
    (('A', 'Clean'), ('B', 'Dirty')): 'Suck',
    (('B', 'Dirty'), ('B', 'Clean')): 'Left',
    (('B', 'Clean'), ('A', 'Dirty')): 'Suck'
}

percepts = []

def table_driven_agent(percept):
    percepts.append(percept)

    if tuple(percepts) in sequence_table:
        return sequence_table[tuple(percepts)]

    return table.get(percept, "NoOp")

print("Vacuum Cleaner Table-Driven Agent")

n = int(input("Enter number of percepts: "))

for i in range(n):
```

```

print(f"\nPercept {i+1}")

location = input("Enter Location (A/B): ").upper()
status = input("Enter Status (Clean/Dirty): ").capitalize()

percept = (location, status)

action = table_driven_agent(percept)

print("Percept History :", percepts)
print("Agent Action   :", action)

```

OUTPUT:

```

... Vacuum Cleaner Table-Driven Agent

Percept 1
Percept History : [('A', 'Dirty')]
Agent Action    : Suck

Percept 2
Percept History : [('A', 'Dirty'), ('B', 'Clean')]
Agent Action    : Left

Percept 3
Percept History : [('A', 'Dirty'), ('B', 'Clean'), ('A', 'Dirty')]
Agent Action    : Suck

Percept 4
Percept History : [('A', 'Dirty'), ('B', 'Clean'), ('A', 'Dirty'), ('B', 'Clean')]
Agent Action    : Left

```

2). Design a Simple Rational Agent by defining PEAS for Autonomous Taxi Environment. Classify environments (fully/partially observable, deterministic/stochastic, episodic/sequential). Implement a simple table-driven agent in Python.

→

CODE:

```

table = {
    ('Green', 'No', 'No'): 'Move Forward',
    ('Green', 'Yes', 'No'): 'Stop',
    ('Red', 'No', 'No'): 'Stop',
    ('Yellow', 'No', 'No'): 'Slow Down',
    ('Green', 'No', 'Yes'): 'Pick Up Passenger',
    ('Red', 'Yes', 'No'): 'Stop',
    ('Yellow', 'Yes', 'No'): 'Stop'
}

percepts = []

```

```

def taxi_agent(percept):
    percepts.append(percept)

    action = table.get(percept, "Wait")

    return action

print("Autonomous Taxi Table-Driven Agent")

n = int(input("Enter number of percepts: "))

for i in range(n):

    print(f"\nPercept {i+1}")

    signal = input("Traffic Signal (Green/Yellow/Red): ").capitalize()
    obstacle = input("Obstacle Present? (Yes/No): ").capitalize()
    passenger = input("Passenger Request? (Yes/No): ").capitalize()

    percept = (signal, obstacle, passenger)

    action = taxi_agent(percept)

    print("Current Percept:", percept)
    print("Agent Action:", action)

```

OUTPUT:

```

... Autonomous Taxi Table-Driven Agent

Percept 1
Current Percept: ('Red', 'Yes', 'Yes')
Agent Action: Wait

Percept 2
Current Percept: ('Green', 'No', 'No')
Agent Action: Move Forward

```

3. Drone implementations:

→

CODE:

```

table = {
    ('A', 'High', 'Full'): 'Engage',
    ('A', 'Low', 'Full'): 'Patrol',
    ('A', 'None', 'Full'): 'Scout',
    ('A', 'High', 'Low'): 'Retreat',
    ('A', 'Low', 'Low'): 'Return to Base',
    ('B', 'High', 'Full'): 'Engage',
    ('B', 'Low', 'Full'): 'Patrol',
    ('B', 'None', 'Full'): 'Scout',
    ('B', 'High', 'Low'): 'Retreat',
    ('B', 'Low', 'Low'): 'Return to Base',
}

```

```
percepts = []

def drone_agent(percept):
    percepts.append(percept)
    return table.get(percept, "Hover")

print("Military Drone Agent")
n = int(input("Enter number of percepts: "))

for i in range(n):
    print(f"\nPercept {i+1}")
    zone = input("Zone (A/B): ").upper()
    threat = input("Threat (High/Low/None): ").capitalize()
    fuel = input("Fuel (Full/Low): ").capitalize()

    percept = (zone, threat, fuel)
    action = drone_agent(percept)

    print("History :", percepts)
    print("Action :", action)
```

OUTPUT:

```
... Military Drone Agent

Percept 1
History : [('A', 'High', 'Full')]
Action  : Engage

Percept 2
History : [('A', 'High', 'Full'), ('B', 'Low', 'Low')]
Action  : Return to Base

Percept 3
History : [('A', 'High', 'Full'), ('B', 'Low', 'Low'), ('A', 'None', 'Full')]
Action  : Scout

Percept 4
History : [('A', 'High', 'Full'), ('B', 'Low', 'Low'), ('A', 'None', 'Full'), ('B', 'High', 'Low')]
Action  : Retreat
```